GPUMODE Lecture Study Notes: Lecture 7 Advanced Quantization

Contents

1 Core Problem the Lecture Addresses

This lecture studies how to make neural network inference faster on GPUs by using quantization without losing too much model quality. The central engineering problem is not simply "store weights in fewer bits." The real problem is:

[Can we reduce memory traffic and arithmetic cost without introducing kernels whose overhead erases the benefit]

The speaker, Charles Hernandez from the PyTorch core team and the `torchao` quantization and pruning team, discusses practical GPU quantization work used in PyTorch case studies such as GPT Fast, SAM Fast, and SDXL Fast. The lecture is especially valuable because it explains the intersection of:

- quantization theory,
- CUDA and Triton kernel behavior,
- PyTorch `torch.compile` and Inductor code generation,
- memory-bound versus compute-bound inference,
- low-bit dtypes such as int8 and int4,
- why some quantized kernels are fast while others are accidentally slower than the unquantized baseline.

The lecture emphasizes that quantization is a trade-off between accuracy and performance:

[Better performance $\iff$ less numerical fidelity, more kernel complexity, or both.]

The PyTorch AO team focuses on taking a model that already works and making it smaller or faster. The humorous but technically accurate framing is:

> Quantization takes a model that works well and makes it work worse, but faster.

The core GPU systems lesson is that the useful quantization scheme depends on whether the model layer is compute-bound or memory-bound.

For compute-bound workloads such as parts of Segment Anything, dynamic int8 quantization can help because int8 arithmetic has higher throughput than floating-point arithmetic.

For memory-bound workloads such as small-batch LLM decoding, weight-only quantization can help because the bottleneck is loading weights from high-bandwidth memory rather than doing arithmetic.

[Compute-bound $\Rightarrow$ reduce arithmetic cost]

[Memory-bound $\Rightarrow$ reduce bytes moved from memory]

The lecture's most important practical message is:

> A quantized model is only faster if the kernel implementation preserves the theoretical advantage of quantization. Low-bit storage alone is not enough.

2 Required Background

2.1 Linear Layers and Matrix Multiplication

Most of the lecture focuses on quantizing linear layers. A linear layer can be written as:

[Y = XW,]

where:

- (X $\in \mathbb{R}^{M \times K}$)$istheactivationmatrix$,
- (W $\in \mathbb{R}^{K \times N}$)$istheweightmatrix$,
- (Y $\in \mathbb{R}^{M \times N}$)$istheoutputmatrix$.

The elementwise formula is:

[$Y_{m,n}$ =======

$\sum_{k=0}^{K-1} X_{m,k} W_{k,n}$.]

For LLM autoregressive decoding with batch size (1), this often degenerates into a vector-matrix product:

[x $\in \mathbb{R}^{1 \times K}$, $\quad W \in \mathbb{R}^{K \times N}$, $\quad y = xW \in \mathbb{R}^{1 \times N}$.]

This special case matters because it changes the optimal GPU parallelization strategy. A full GEMM uses tensor cores efficiently when (M), (N), and (K) are sufficiently large. A batch-size-one vector-matrix product often becomes memory-bound and may not use tensor cores efficiently.

2.2 GPU Execution Model

A GPU kernel is executed by many threads grouped into thread blocks. Thread blocks are scheduled onto streaming multiprocessors, or SMs.

- A **thread** executes scalar instructions.
- A **warp** is usually (32) threads executing in SIMT style.
- A **thread block** is a group of threads that can cooperate through shared memory.
- A **grid** is the full set of thread blocks launched by a kernel.
- An **SM** schedules warps, manages registers and shared memory, and issues instructions.

A standard tiled matrix multiplication assigns each program or thread block a tile of the output matrix:

[$Y_{m:m+B_M,\ n:n+B_N}$.]

It then loops over the (K)-dimension in chunks:

[$Y_{\text{tile}} \leftarrow Y_{\text{tile}} + X_{\text{tile}} W_{\text{tile}}$.]

The performance depends heavily on:

- coalesced global memory loads,
- reuse of tiles in shared memory or registers,
- tensor core eligibility,
- occupancy,
- register pressure,
- arithmetic intensity.

2.3 Memory Hierarchy

A simplified GPU memory hierarchy is:

[Registers $\rightarrow$ Shared Memory / L1 $\rightarrow$ L2 Cache $\rightarrow$ HBM / Global Memory.]

The closer memory is to the SM, the lower its latency and the smaller its capacity. Quantization often improves performance by reducing HBM traffic:

[Bytes moved $=$

number of elements $\times$bytes per element.]

For common dtypes:

[int4 $= 0.5$ bytes, int8 $= 1$ byte, fp16/bf16 $= 2$ bytes, fp32/int32 $= 4$ bytes.]

So converting weights from bf16 to int4 can reduce the weight bandwidth by a factor of:

[$2_{\overline{0.5=4.}}$

But this does not automatically mean the kernel is $(4\times) faster, because the kernel may now need extra work for dequant$

2.4 Compute-Bound Versus Memory-Bound Kernels

A kernel is compute-bound when arithmetic throughput is the bottleneck. A kernel is memory-bound when memory bandwidth is the bottleneck.

The roofline model expresses this using arithmetic intensity:

[Arithmetic Intensity $=$

$\text{FLOPs}_{\overline{\text{Bytes moved.}}]}$

The attainable performance is approximately:

[Performance $\leq \min$ (Peak Compute, ; Memory Bandwidth $\times$ Arithmetic Intensity) .]

If arithmetic intensity is low, reducing bytes moved is valuable. If arithmetic intensity is high, reducing arithmetic cost is valuable.

2.5 Quantization Basics

Quantization maps high-precision values to a smaller set of discrete low-bit values. A simple affine quantizer is:

[q =

clip $\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right)$,]

where:

- (x) is the original floating-point value,
- (q) is the quantized integer value,
- (s) is the scale,
- (z) is the zero point,
- $(\text{q}_{\min}) and (q_{\max}) define the integer range.$

Dequantization reconstructs an approximate floating-point value:

[=======

s(q-z).]

For symmetric quantization, the zero point is usually omitted or fixed at zero:

[q =

clip $\left(\text{round}\left(\frac{x}{s}\right), -q_{\max}, q_{\max}\right)$, $\quad \hat{x}$ =======

sq.]

For int8 symmetric quantization, a common choice is:

[$\text{q}_{\max} = 127, \quad s =$

$\max_i |x_i|_{\overline{127.]}}$

For signed int4 symmetric quantization, a common representable range is:

[$\text{q} \in -8, -7, \ldots, 7.$]

Because int4 has only (16) possible values, scale choice and grouping strategy become much more important than in int8.

3 Main Concepts

3.1 Ambiguity of the Phrase "int4 Quantization"

The lecture highlights that phrases such as "int4 quantization" are often ambiguous. A better notation specifies what happens to activations, weights, and accumulation.

For example:

[A16W4]

means:

- activations use (16)-bit floating point, usually fp16 or bf16,
- weights use (4)-bit quantization,
- accumulation may still happen in fp32 or another dtype.

Similarly:
[A8W8]
means activations and weights are both quantized to int8.
A fully explicit notation would include:
[A*b$_a$W $*$ b_wAcc $*$ b $*$ acc,]
where:

- (b$_a$)$istheactivationprecision$,
- (b$_w$)$istheweightprecision$,
- (b$_{\text{acc}}$)$istheaccumulationprecision$.

For example:
[A8W8Acc32]
means int8 activations, int8 weights, and int32 accumulation.
This matters because two methods both called "int4 quantization" may be completely different:

- A16W4 weight-only quantization,
- A8W4 dynamic or static quantization,
- fp16 activations with packed int4 weights,
- int4 weights dequantized before multiplication,
- mixed int4-by-bf16 arithmetic inside a custom kernel,
- int4 storage with fp32 accumulation.

The performance and accuracy of these schemes can differ dramatically.

3.2 Dynamic Quantization

Dynamic quantization quantizes both weights and activations at runtime. The lecture's main dynamic quantization case is A8W8.

The conceptual computation is:

[Y =

XW.]

We approximate:

[$X \approx s_X X_q, \qquad W \approx s_W W_q,$]

where:

[$X_q \in \mathbb{Z}^{M \times K}, \qquad W_q \in \mathbb{Z}^{K \times N}.$]

Then:

[Y =

$XW \approx (s_X X_q)(s_W W_q).$]

If the scales are broadcastable over the output dimensions, this becomes:

[$Y \approx s_X s_W \, (X_q W_q) \, .$]

The integer matrix multiplication is:

[$Z_{m,n}$ =======

$\sum_{k=0}^{K-1} (X_q) * m, k(W_q) * k, n.$]

Because each term is int8 times int8, the products and sums are accumulated in int32:

[$Z_{m,n} \in \mathbb{Z}_{32}.$]

The final output is rescaled:

[$Y_{m,n} \approx s_X(m) s_W(n) Z_{m,n}.$]

The scales may be:

- per-tensor,
- per-token,
- per-channel.

The important restriction is that the scale dimensions must survive the matrix multiplication. Since the (K)-dimension is reduced away, scales that vary over (K) cannot be applied after the matmul output without changing the computation.

For $(X \in \mathbb{R}^{M \times K}) and (W \in \mathbb{R}^{K \times N})$:

[$Y \in \mathbb{R}^{M \times N}.$]

So a scale depending on (M) or (N) can be applied to (Y), but a scale depending on (K) generally cannot be applied after reduction.

3.2.1 Why Dynamic Quantization Helps Compute-Bound Models

Dynamic quantization helps when the multiplication is the bottleneck. The lecture mentions Segment Anything as an example where dynamic quantization can work well.

The performance reason is:

[int8 multiply throughput

>

bf16 multiply throughput]

on relevant GPU hardware. If a layer is dominated by multiplication, A8W8 may improve throughput.

However, the dynamic quantization pipeline has overhead:

1. compute activation scales,
2. quantize activations,
3. perform int8 matrix multiplication,
4. accumulate into int32,
5. rescale the output.

If the layer is not compute-bound, this overhead can dominate.

3.3 Weight-Only Quantization

Weight-only quantization keeps activations in high precision while quantizing only the weights.

For A16W8:

[X $\in$ bf16 or fp16, $W_q \in$ int8.]

For A16W4:

[X $\in$ bf16 or fp16, $W_q \in$ int4.]

The weight is represented as:

[W $\approx s_W W_q$.]

The computation becomes:

[Y =

XW $\approx X(s_W W_q)$.]

Depending on the kernel, this may be implemented as:

[$Y_{m,n}$ =======

$s_W(n) \sum_{k=0}^{K-1} X_{m,k}(W_q)_{k,n}$,]

or as a dequantize-then-multiply sequence:

[$_{k,n}$ ==========

$\mathrm{s}_W(n)(W_q)_{k,n}, \qquad Y = X\hat{W}.$]

The key benefit is not cheaper activation arithmetic. The key benefit is lower weight bandwidth.

For bf16 weights:

[bytes per weight = 2.]

For int8 weights:

[bytes per weight = 1.]

For int4 weights:

[bytes per weight = 0.5.]

Thus, the ideal memory traffic reduction is:

[bf16 $\rightarrow$ int8 : 2$\times$, bf16 $\rightarrow$ int4 : 4 $\times$.]

This is especially useful for LLM decoding at batch size (1), where each generated token may require streaming large weight matrices from memory.

3.3.1 Why Weight-Only Quantization Helps Memory-Bound Models

In small-batch LLM inference, the activation vector is small and already resident or cheap to load relative to the weights. The large cost is repeatedly loading weights.

For a single token:

[x $\in \mathbb{R}^{1\times K}, \qquad W \in \mathbb{R}^{K\times N}$.]

The number of activation elements is:

[K.]

The number of weight elements is:

[KN.]

For large (N), the weight matrix dominates memory movement.

So the priority is:

[load weights as cheaply as possible.]

This is why the lecture says that for memory-bound LLM inference, weight-only quantization can be better than dynamic quantization. Quantizing activations may add extra work and intermediate tensors without addressing the real bottleneck.

3.4 Quantization Granularity

Quantization granularity determines how many values share one scale.

3.4.1 Per-Tensor Quantization

One scale for the entire tensor:

[s = $\max_{i,j} |W_{i,j}|_{\overline{q_{\max}.}}$

This is cheap because it stores only one scale, but it is sensitive to outliers.

3.4.2 Per-Channel Quantization

One scale per output channel:

[$s_n = \frac{\max_k |W_{k,n}|}{q_{\max}}$.]

Then:

[$W_{k,n} \approx s_n(W_q)_{k,n}$.]

This is common for linear layers because each output channel can have a different dynamic range.

3.4.3 Per-Token Quantization

For activations, one scale may be computed per token or row:

[$s_m = \frac{\max_k |X_{m,k}|}{q_{\max}}$.]

Then:

[$X_{m,k} \approx s_m(X_q)_{m,k}$.]

3.4.4 Group-Wise Quantization

In group-wise quantization, channels or vector elements are split into groups. Each group gets its own scale.

Let group size be (G). For a weight matrix column (n), group (g) covers:

[$k \in gG, gG+1, \ldots, (g+1)G-1$.]

The group scale is:

[$s_{g,n}$ =======

$\max_{k \in \text{group}(g)} |W_{k,n}|_{\overline{q_{\max}.}}$

The approximation is:

[$W_{k,n} \approx s_{g(k),n}(W_q)_{k,n}$.]

Smaller groups increase accuracy because they reduce the effect of outliers, but they also increase scale metadata and rescaling overhead.

[smaller group ⇒ more scales ⇒ higher fidelity ⇒ more metadata and kernel work.]

3.5 Outliers and Dynamic Range

A major problem in quantization is that a few large values can determine the scale.

Suppose a tensor has values mostly in:

[[-0.2, 0.2],]

but has one outlier at:

[100.]

For symmetric int8 quantization:

[s = $100_{\overline{127.]}}$

The spacing between adjacent quantized levels is about:

[$\Delta = s \approx 0.787.$]

Most values in ([-0.2,0.2]) collapse to zero. This destroys information.

Using finer granularity can help. If the outlier is isolated to one channel or group, other channels or groups can use smaller scales.

> More local quantization scales reduce the damage caused by outliers, but every additional scale has a storage and computation cost.

3.6 Quantization-Aware Training

Quantization-aware training, or QAT, trains the model while simulating quantization effects.

The forward pass uses fake quantization:

[x $\rightarrow q =$

round $\left(\frac{x}{s}\right) \rightarrow \hat{x}$ =======

sq.]

The problem is that rounding is not differentiable:

[$\mathrm{d}_{\overline{\frac{d}{dx}\,\mathrm{round}(x)=0 \quad \text{almost everywhere.}]}}$

A common workaround is the straight-through estimator:

[$\mathrm{d}_{\overline{\frac{d}{dx}\,\mathrm{round}(x)\approx 1.]}}$

This allows gradients to pass through the fake quantization operator as if it were identity.

QAT is useful when post-training quantization causes too much accuracy loss.

3.7 GPTQ and Calibration-Based Quantization

The lecture mentions GPTQ as a method needed to make int4 quantization accurate enough for GPT Fast.

GPTQ is a post-training quantization method that uses calibration data. It estimates how important

each weight direction is by observing activations.

For a linear layer:

[Y = XW.]

The squared error caused by quantizing (W) is approximately:

[|X(W-)|$_2^2$.]

This can be written using an activation covariance or Hessian-like matrix:

[H = X$^\top X$.]

Then:

[|X(W-)|$_2^2$ ==========

(W-)$^\top H(W - \hat{W})$.]

GPTQ quantizes weights while compensating remaining unquantized weights to reduce output error. The rough idea is:

[W $\rightarrow \hat{W}$, while minimizing $(W - \hat{W})^\top H(W - \hat{W})$.]

This is more expensive than simple int8 quantization. It requires calibration data and may take significant time, but it can preserve quality at lower precision such as int4.

4 Hardware-Level Explanation

4.1 Why Intermediate Materialization Can Destroy Memory Benefits

In dynamic quantization, an int8-by-int8 matmul naturally accumulates into int32:

[int8 $\times$int8 $\rightarrow$ int32 accumulation.]

If the kernel stores the int32 output matrix (Z) to global memory before rescaling, then the intermediate tensor costs:

[4MN bytes.]

The unquantized bf16 output costs:

[2MN bytes.]

So a bad dynamic quantization implementation can increase peak memory even though the inputs are quantized.

The desired computation is:

[Y =

s$_X s_W Z$.]

The bad implementation is:

[Z = X$_q W_q$ stored to HBM as int32,]

then later:

[Y = $s_X s_W Z$.]

The better implementation fuses the rescale into the matmul epilogue:

[Y =

store $(s_X s_W \sum_k X_{q,m,k} W_{q,k,n})$.]

In this case, each thread or program may still use int32 registers internally, but it stores the final bf16 or fp32 result rather than materializing an int32 tensor in global memory.

> Temporary int32 accumulation in registers is normal. Storing a full int32 output tensor to global memory is often the real performance and memory problem.

4.2 Epilogue Fusion

In matrix multiplication kernels, an epilogue is the computation applied to the accumulator before storing the result.

For a normal matmul:

[C = AB.]

For a quantized dynamic matmul:

[C = $s_A s_B (A_q B_q)$.]

The scale multiply should be part of the epilogue:

[$C_{m,n}$ =======

cast $(s_A(m) s_B(n)\, \text{acc}_{m,n})$.]

If this operation is not fused, PyTorch may create an intermediate output tensor and then launch another kernel for the multiply. That costs extra memory bandwidth and launch overhead.

The lecture explains that adding this fused multiplication in Triton is conceptually simple, but getting `torch.compile` and Inductor to automatically fuse the pattern was difficult. A hard-coded option was added for a pattern like int8 matmul followed by multiplication.

The relevant system lesson is:

[Mathematical simplicity $\not\Rightarrow$ compiler automatically emits the desired fused kernel.]

4.3 Prologue Casts for Weight-Only Quantization

A prologue is work done to inputs before the main multiply-accumulate loop.

For int8 weight-only quantization, one tempting implementation is:

[$W_q \in \text{int8} \to \text{cast}(W_q) \in \text{bf16} \to X \cdot W_q$.]

This is a prologue cast. It appears simple, but the lecture shows that a naive Triton matmul with this cast can be much slower than the unquantized baseline.

Reasons include:

- extra conversion operations,
- extra scaling operations,
- tensor core shape constraints,
- poor fit for batch-size-one vector-matrix multiplication,
- insufficient parallelism when the matrix dimension used for tiling is too small.

4.4 Tensor Core Tile Shape Constraints

Triton matmul kernels often use block sizes such as:

[$\mathrm{B}_M, \quad B_N, \quad B_K$.]

For tensor-core-friendly kernels, the tile dimensions are often constrained. The lecture mentions that some Triton configurations effectively require dimensions such as ($\mathrm{B}_M \geq 16$).

For batch size (1), (M=1). If the kernel is forced into a tile shape intended for ($\mathrm{M} \geq 16$), $much of the work assignment is inefficient.$

The mismatch is:

[M=1 but kernel assumes a tile shape useful for $\mathrm{M} \geq 16$.]

This is one reason a standard matmul template may perform poorly for LLM decoding.

4.5 Vector-Matrix Parallelism for Batch Size One

For batch size one:

[$\mathrm{x} \in \mathbb{R}^{1\times K}, \qquad W \in \mathbb{R}^{K\times N}$.]

The output is:

[y_n ===
$\sum_{k=0}^{K-1} x_k W_{k,n}$.]

Each output element (y_n)$depends on one column of (W). Therefore, the computation can be parallelized over columns :$

[program id $\leftrightarrow$ n.]

Each Triton program computes one or a small number of output columns.

This avoids the standard tensor-core matmul path and instead uses an embarrassingly parallel reduction over the (K)-dimension.

The lecture describes a PyTorch expression that looks like a manual matmul:

[Y =
$\sum_k X_{m,k} W_{k,n}$.]

Using broadcasting, this resembles:

[(X[:, :, None] $\cdot W$[None, :, :]). sum(dim = 1).]

When compiled with `torch.compile`, this pattern produced a surprisingly fast kernel for int8 weight-only batch-size-one inference.

4.6 Why Batch Size Greater Than One Is Harder

When (M=1), a program can map directly to a column (n). But when (M>1), the output has two dimensions:

[$Y \in \mathbb{R}^{M \times N}$.]

Now the kernel must parallelize over both:

[m and n.]

The problem becomes closer to a real GEMM, and the simple column-parallel trick is no longer enough. The arithmetic intensity changes because the same weight column may be reused across multiple batch elements.

For (M=1), weight loading dominates:

[memory-bound.]

For larger (M), weight reuse increases:

[more compute-bound.]

The ideal kernel for (M=1) may be poor for (M>1).

4.7 int4 Packing and Unpacking

PyTorch and Triton do not generally expose a convenient native int4 dtype for this style of GPU programming. So int4 values are usually packed into int8 or uint8 containers.

Two signed or unsigned (4)-bit values fit into one byte:

[byte ==========

low nibble + 16 ·high nibble.]

If the byte is (b), then:

[$q_0 = b\ 15,$]

[$q_1 = b \gg 4.$]

For signed int4, values may need conversion from unsigned range ([0,15]) to signed range ([-8,7]):

[q_{signed} ==========

$$\begin{cases} q, & q < 8, \ q - 16, \\ q \geq 8. \end{cases}$$]

Packing introduces a new kernel cost. A low-bit kernel must:

1. load a byte,

2. extract one or both nibbles,

3. convert to signed or floating-point representation,

4. apply scales,

5. multiply by activations,

6. accumulate.

The ideal kernel would avoid expanding int4 all the way to bf16 before multiplication, but that requires lower-level dtype and bit manipulation support.

4.8 L2 Cache Issue with Packed Low-Bit Data

The lecture points out a subtle cache behavior problem. If packed int4 weights are loaded from HBM, the L2 cache contains the packed representation.

Suppose multiple threads need the same unpacked values. Unless the kernel explicitly arranges sharing, each thread may have to unpack the same data independently.

[L2 cache contains packed bytes $\neq$ L2 cache contains unpacked int4 values.]

Therefore, the unpack overhead can be repeated. This is one reason int4 kernels are hard to make fast in Triton.

4.9 Triton Strengths and Limits

Triton is excellent when the operation is close to a standard block-structured tensor program and uses supported dtypes. The lecture praises Triton for:

- quick iteration,
- easy epilogue and prologue modifications,
- writing fused kernels in a Python-like language,
- allowing `torch.compile` to generate fast kernels,
- getting a large fraction of optimal performance with much less effort than CUDA.

But Triton becomes less ideal when:

- the dtype is non-standard, such as int4 or fp4,
- manual packing and unpacking dominate,
- bit-level manipulation is required,
- a custom memory layout is needed,
- the kernel must exploit hardware-specific low-level instructions,
- batch-size edge cases break standard tile assumptions.

The lecture's practical boundary is:

[int8 ⇒ Triton and torch.compile can work very well.]

[int4 ⇒ custom CUDA or specialized kernels often become necessary.]

5 Mathematical Reasoning

5.1 Uniform Quantization Formula

Given a tensor (T), symmetric per-tensor quantization uses:

[s = $\max_i |T_i|_{\overline{q_{\max}.}}$

Then:

[$Q_i = \text{clip}\left(\text{round}\left(\frac{T_i}{s}\right), -q_{\max}, q_{\max}\right)$.]

The reconstruction is:

[$_i = sQ_i$.]

The quantization error is:

[$\epsilon_i = T_i - \hat{T}_i$.]

For a uniform quantizer with step size (s), if values are not clipped and are distributed smoothly within bins, the error magnitude is bounded by:

[$|\epsilon_i| \leq \frac{s}{2}$.]

This shows why smaller scales improve fidelity, but the representable range is limited by:

[[-$q_{\max}s, q_{\max}s$].]

5.2 Dynamic Quantized Linear Layer

Let:

[X $\approx s_X X_q, \qquad W \approx s_W W_q$.]

Then:

[Y = XW $\approx s_X s_W X_q W_q$.]

For per-token activation scales ($s_X(m)$)$andper - channelweightscales(s_W(n))$:

[$Y_{m,n} \approx s_X(m)s_W(n)\sum_{k=0}^{K-1}(X_q) * m, k(W_q) * k, n$.]

The inner sum is performed in int32:

[$Z_{m,n}$ =======

$\sum_{k=0}^{K-1}(X_q) * m, k(W_q) * k, n$.]

So:

[$Y_{m,n} \approx s_X(m)s_W(n)Z_{m,n}$.]

The storage choice for (Z) is critical. If (Z) is stored as int32, memory increases. If scaling is fused, (Z) remains a register-level accumulator and the stored output can be bf16 or fp32.

5.3 Weight-Only Linear Layer

For weight-only quantization:

[$\mathrm{W} \approx s_W W_q$.]

Then:

[$\mathrm{Y}_{m,n} \approx \sum_{k=0}^{K-1} X_{m,k} s_W(n)(W_q)_{k,n}$.]

If $(\mathrm{s}_W(n)) is independent of (k), it can be pulled outside the reduction$:

[$\mathrm{Y}_{m,n} \approx s_W(n) \sum_{k=0}^{K-1} X_{m,k}(W_q)_{k,n}$.]

If group-wise scales are used, $(\mathrm{s}_W) depends on (k)$:

[$\mathrm{Y}_{m,n} \approx \sum_{k=0}^{K-1} X_{m,k} s_W(g(k), n)(W_q)_{k,n}$.]

In that case, the scale multiply must occur inside or near the reduction loop.

5.4 Memory Traffic Analysis

For a linear layer with weight matrix $(\mathrm{W} \in \mathbb{R}^{K \times N}), the weight memory traffic is approximately$:

[$\mathrm{B}_W = KN \cdot b_W,$]

where $(\mathrm{b}_W) is bytes per weight.$

For bf16:

[$\mathrm{B}_W^{\mathrm{bf16}} = 2KN$.]

For int8:

[$\mathrm{B}_W^{\mathrm{int8}} = KN$.]

For packed int4:

[$\mathrm{B}_W^{\mathrm{int4}} = \frac{1}{2}KN$.]

The theoretical bandwidth reduction relative to bf16 is:

[$\mathrm{B}_W^{\mathrm{bf16}} \frac{}{B_W^{\mathrm{int8}}} ==========$

2,]

[$\mathrm{B}_W^{\mathrm{bf16}} \frac{}{B_W^{\mathrm{int4}}} ==========$

4.

]

However, the real runtime includes overhead:

[$\mathrm{T}_{\mathrm{quantized}}$ ==========

$\mathrm{T}_{\mathrm{load\ quantized\ weights}} + T_{\mathrm{unpack}} + T_{\mathrm{dequantize}} + T_{\mathrm{multiply\ accumulate}} + T_{\mathrm{rescale/store}}$.]

Quantization helps only if:

[$T_{\text{quantized}} < T_{\text{bf16 baseline}}$.]

5.5 Arithmetic Intensity of Batch-Size-One Linear Inference

For $(x \in \mathbb{R}^{1\times K}) and (W \in \mathbb{R}^{K\times N}), the approximate number of multiply-add operations is$:

[2KN.]

The weight bytes for bf16 are:

[2KN.]

Ignoring activation and output traffic, arithmetic intensity is:

[$I_{\text{bf16}} \approx \frac{2KN}{2KN}$ ==========

1 FLOP/byte.]

For int4 weights:

[B_W^{int4} ==========

$1_{\overline{2KN.]}}$

The ideal arithmetic intensity becomes:

[$I_{\text{int4}} \approx \frac{2KN}{\frac{1}{2}KN}$ ==========

4 FLOPs/byte.]

So int4 can move the operation closer to compute-bound behavior, but only if unpacking and dequantization do not dominate.

5.6 Accumulation Overflow

For int8 multiplication:

[a,b $\in [-128, 127]$.]

The maximum product magnitude is approximately:

[$|ab| \leq 128^2 = 16384$.]

Summing over (K) terms gives worst-case magnitude:

[$|Z| \leq K \cdot 16384$.]

For (K=4096):

[$|Z| \leq 4096 \cdot 16384$ ==========

67,108,864.]

This exceeds int16 range. Therefore int32 accumulation is natural and often hardware-supported.

This explains why:

[int8 $\times$int8 $\rightarrow$ int32 accumulation]

is standard.

Accumulating into fp64 is usually unnecessary for deep learning inference and is poorly matched to hardware throughput. In practice, engineers choose the smallest accumulation dtype that avoids unacceptable overflow or accuracy loss.

6 Code Walkthrough

6.1 Simple Symmetric Quantization in PyTorch

```
import torch

def symmetric_quantize_int8(x, dim=None, eps=1e-8):
qmax = 127.0

'''
if dim is None:
    max_abs = x.abs().max()
else:
    max_abs = x.abs().amax(dim=dim, keepdim=True)

scale = torch.clamp(max_abs / qmax, min=eps)
q = torch.round(x / scale).clamp(-127, 127).to(torch.int8)
return q, scale
'''

def symmetric_dequantize(q, scale):
return q.float() * scale
```

This code implements symmetric int8 quantization.

The line:

```
max_abs = x.abs().amax(dim=dim, keepdim=True)
```

chooses the quantization granularity. If `dim=None`, the entire tensor shares one scale. If `dim` is specified, the scale is computed along that dimension.

The scale is:

$$[\ \mathrm{s} = \max|x|_{\overline{127.}}$$

The quantized tensor is:

$$[\ \mathrm{q} = \mathrm{round}\left(\frac{x}{s}\right).]$$

The hardware implication is that the resulting `q` uses one byte per element instead of two bytes for bf16 or four bytes for fp32. But dequantization creates floating-point values again, so the performance depends on whether dequantization is fused into a useful kernel.

6.2 Dynamic Quantized Linear Layer

```
def dynamic_quantized_linear(x, w):
# x: [M, K], floating point activation
# w: [K, N], floating point weight

```
x_q, x_scale = symmetric_quantize_int8(x, dim=1)
w_q, w_scale = symmetric_quantize_int8(w, dim=0)

acc = torch.matmul(x_q.to(torch.int32), w_q.to(torch.int32))

y = acc.float() * x_scale * w_scale
return y
```
```

Conceptually, this computes:

[$Y_{m,n} \approx s_X(m) s_W(n) \sum_k (X_q) * m, k(W_q) * k, n.$]

However, this Python version is not an efficient GPU implementation because it explicitly materializes:

[acc $\in$ int32$^{M \times N}$.]

The lecture's key optimization is to avoid writing this full int32 tensor to global memory. The scale multiply should be fused into the matmul epilogue.

A better kernel-level implementation conceptually does:

```
# Pseudocode inside a matmul kernel

acc = int32_accumulator_tile()

for k_block in k_blocks:
x_tile = load_int8_x_tile()
w_tile = load_int8_w_tile()
acc += dot_int8_int8(x_tile, w_tile)

scale_tile = load_x_scale() * load_w_scale()
out_tile = acc * scale_tile
store(out_tile)
```

The important hardware behavior is:

- acc lives in registers or local accumulator fragments,
- the kernel stores the final scaled output,
- no full int32 matrix is stored to HBM,
- memory traffic and peak memory are reduced.

6.3 Triton-Style Dynamic Quantized Matmul With Epilogue

```
# Pseudocode, not a complete standalone Triton kernel.

@triton.jit
def int8_matmul_epilogue_scale(
Xq, Wq, X_scale, W_scale, Y,
M: tl.constexpr, N: tl.constexpr, K: tl.constexpr,
BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
):
pid_m = tl.program_id(0)
pid_n = tl.program_id(1)

```
offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
offs_k = tl.arange(0, BLOCK_K)

acc = tl.zeros((BLOCK_M, BLOCK_N), tl.int32)

for k0 in range(0, K, BLOCK_K):
    x = tl.load(Xq + offs_m[:, None] * K + (k0 + offs_k[None, :]))
    w = tl.load(Wq + (k0 + offs_k[:, None]) * N + offs_n[None, :])
    acc += tl.dot(x, w, out_dtype=tl.int32)

sx = tl.load(X_scale + offs_m)
sw = tl.load(W_scale + offs_n)

y = acc.to(tl.float32) * sx[:, None] * sw[None, :]
tl.store(Y + offs_m[:, None] * N + offs_n[None, :], y)
```
```

The key idea is that scale multiplication occurs before the store. This is an epilogue. Without this fusion, the program may store `acc` as int32, then launch a second kernel to multiply by scales.

The output tile indexing is:

$[\ \mathrm{m} = \mathtt{pid}_m \cdot B_M + i, \qquad n =$

6.4 Naive Weight-Only Linear Layer

```
def weight_only_linear_naive(x, w_q, w_scale):
# x: [M, K], bf16 or fp16
# w_q: [K, N], int8
# w_scale: [N] or broadcastable scale

```
w_deq = w_q.to(x.dtype) * w_scale
return x @ w_deq
```
```

This is mathematically correct, but inefficient. It materializes the dequantized weight matrix:

$[\ = \mathrm{s}_W W_q \cdot]$

If () is stored in bf16, then the memory benefit of int8 storage is partially lost. The better implementation loads quantized weights and performs conversion and scaling inside the matmul kernel.

6.5 Weight-Only Vector-Matrix Trick for Batch Size One

For batch size one, the computation is:

[y_n ===

$\mathrm{s}_W(n) \sum_k x_k (W_q)_{k,n}$.]

A PyTorch expression that exposes this structure is:

```
def weight_only_vecmat_torch_compile_pattern(x, w_q, w_scale):
# x: [1, K]
# w_q: [K, N]
# w_scale: [N]

```
prod = x[:, :, None] * w_q.to(x.dtype)[None, :, :]
y = prod.sum(dim=1)
y = y * w_scale
return y
```
```

This looks wasteful in eager PyTorch because the broadcasted product would have shape:

[$1 \times K \times N$.]

But with `torch.compile`, the compiler can avoid materializing that full tensor and generate a fused kernel.

The computational mapping is:

[program id n $\Rightarrow$ compute output y_n.]

Each program reads:

[$\mathrm{x}_0, x_1, \ldots, x_{K-1}$]

and one column:

[$\mathrm{W}_{0,n}, W_{1,n}, \ldots, W_{K-1,n}$.]

Then it reduces:

[$\mathrm{y}_n = \sum_k x_k W_{k,n}$.]

This is efficient for batch size one because the output columns are independent. It avoids tensor core tile constraints that are poorly matched to (M=1).

6.6 int4 Packing

```
def pack_uint4(q):
# q: uint8 tensor whose values are in [0, 15]
```

```
# The last dimension must be even.
q0 = q[..., 0::2]
q1 = q[..., 1::2]
packed = q0 | (q1 << 4)
return packed.to(torch.uint8)

def unpack_uint4(packed):
q0 = packed & 0x0F
q1 = (packed >> 4) & 0x0F

```
out_shape = list(packed.shape)
out_shape[-1] *= 2
out = torch.empty(out_shape, device=packed.device, dtype=torch.uint8)

out[..., 0::2] = q0
out[..., 1::2] = q1
return out
```
```

The low nibble is:

[$q_0 = b \ 15$.]

The high nibble is:

[$q_1 = (b \gg 4) \ 15$.]

The packing layout matters. If the kernel processes contiguous values along the (K)-dimension, then packing should preserve contiguity along that dimension. Otherwise, the kernel may load bytes but use only half of each byte, wasting bandwidth.

6.7 Signed int4 Conversion

```
def uint4_to_int4(q):
# q is uint8 with values in [0, 15].
q = q.to(torch.int8)
return torch.where(q >= 8, q - 16, q)
```

This maps:

[$0,\ldots,7 \mapsto 0,\ldots,7,$]

[$8,\ldots,15 \mapsto -8,\ldots,-1$.]

A fused CUDA kernel would avoid creating full intermediate tensors for unpacked values. Instead, it would unpack, convert, multiply, and accumulate in registers.

6.8 Representative CUDA-Like int4 Weight-Only Kernel Skeleton

```
// Pseudocode only: simplified CUDA-style kernel for A16W4 batch-size-one.
extern "C" **global**
void a16w4_vecmat_kernel(
```

```
const half* **restrict** x,          // [K]
const unsigned char* **restrict** w, // packed int4 [K * N / 2]
const half* **restrict** scales,     // [N]
half* **restrict** y,                // [N]
int K,
int N
) {
int n = blockIdx.x * blockDim.x + threadIdx.x;
if (n >= N) {
return;
}

```
float acc = 0.0f;

for (int k = 0; k < K; ++k) {
    int linear = k * N + n;
    int byte_idx = linear >> 1;
    int is_high = linear & 1;

    unsigned char byte = w[byte_idx];
    int q = is_high ? ((byte >> 4) & 0x0F) : (byte & 0x0F);

    if (q >= 8) {
        q -= 16;
    }

    float x_val = __half2float(x[k]);
    acc += x_val * static_cast<float>(q);
}

float scale = __half2float(scales[n]);
y[n] = __float2half(acc * scale);
```

}
```

This skeleton is intentionally simple. A production kernel would use:

- vectorized loads,
- multiple output columns per thread block,
- warp-level reductions,
- shared memory or cache-aware reuse,
- better memory layout for packed weights,
- specialized instructions when available,
- careful register management.

The skeleton shows the key operations:

1. map one thread to one output column (n),
2. iterate over (K),
3. load a packed byte,
4. extract the correct nibble,
5. convert signed int4,
6. multiply by activation,
7. accumulate in fp32,
8. apply scale,
9. store output.

The bottleneck is that every iteration performs bit manipulation and scalar conversion. A highly optimized kernel must make these operations cheap relative to the saved memory bandwidth.

7 Performance Intuition

7.1 Why Dynamic Quantization Improved SAM

Segment Anything has compute-heavy components. For such workloads, replacing bf16 multiplication with int8 multiplication can improve throughput.

The useful change is:

[bf16 $\times$bf16 $\rightarrow$ int8 $\times$ int8.]

If the kernel fuses the scale epilogue, dynamic quantization can reduce runtime and avoid increasing peak memory.

The bad version materializes int32 output:

[$X_q W_q \rightarrow$ stored int32 tensor.]

The good version fuses:

[$X_q W_q \rightarrow$ scale $\rightarrow$ store final output.]

7.2 Why Dynamic Quantization Can Hurt LLM Decoding

In LLM decoding with batch size one, the bottleneck is often loading the weights. Quantizing activations dynamically adds overhead:

- compute activation min or max,
- compute scale,
- quantize activation,

- possibly materialize activation int8,
- execute int8 matmul,
- rescale.

But the activation is small relative to the weights. Therefore, dynamic quantization may solve the wrong problem.

For $(\mathrm{x} \in \mathbb{R}^{1\times K}), activation traffic is roughly$:

[O(K).]

Weight traffic is:

[O(KN).]

When (N) is large:

[$\mathrm{O(KN)} \gg O(K)$.]

Thus, reducing weight traffic is more important.

7.3 Why Weight-Only int8 Can Be Fast

Weight-only int8 halves weight memory traffic relative to bf16:

[$2\mathrm{KN} \rightarrow KN$.]

The arithmetic has some additional work, but for memory-bound kernels, saved bandwidth can dominate.

The ideal weight-only kernel does not materialize dequantized weights. It performs:

[load int8 $\rightarrow$ convert $\rightarrow$ multiply $\rightarrow$ accumulate $\rightarrow$ scale/store]

inside one fused kernel.

7.4 Why a Standard Matmul Kernel Was Slow for A16W8

The lecture reports that a straightforward Triton matmul with a cast from int8 to bf16 performed very badly for weight-only quantization.

The reasons are systems-level rather than mathematical:

- the kernel performs extra conversion and scale work,
- the standard matmul template is designed for larger tiles,
- batch size one does not provide enough (M)-dimension work,
- tensor core constraints may force unsuitable tile sizes,
- the computation is memory-bound but the kernel acts like a general GEMM.

The lesson is:

[Correct kernel $\neq$ fast kernel.]

7.5 Why the Broadcast-Sum Pattern Worked

The broadcast-sum pattern:

[Y = (X[:, :, None] $\cdot W$[None, :, :]). sum(dim = 1)]

lets the compiler generate a kernel closer to the actual batch-size-one structure.

Instead of assigning a ($B_M \times B_N$)$tile, the kernel can assign work over output columns$:

[n = program id.]

This is better matched to vector-matrix inference.

7.6 Why int4 Is Much Harder Than int8

int8 is a real dtype supported by PyTorch, Triton, and hardware paths. int4 is often a storage format rather than a native arithmetic dtype.

The int4 path adds:

[packing + unpacking + sign conversion + scale handling + custom layout concerns.]

If the kernel expands int4 to bf16 too early, it destroys much of the advantage.

The ideal operation would be closer to:

[int4 storage → low-level mixed multiply → accumulate]

rather than:

[int4 storage → unpack → bf16 → multiply.]

This is why custom CUDA kernels can outperform Triton for int4.

7.7 Accuracy Versus Speed

Quantization decisions must consider accuracy. The lecture mentions multiple evaluation modes:

- COCO validation for vision models,
- perplexity for language models,
- HellaSwag or other benchmark tasks,
- human or qualitative checks,
- layer-by-layer sensitivity analysis.

Perplexity is useful because it changes smoothly as quantization settings change. But perplexity is not always directly interpretable as user-visible quality.

A useful engineering view is:

[evaluation metric ==========

CI check for serious degradation + guide for trade-off exploration.]

For production model quality, human preference or task-specific evaluation may still be necessary.

8 Common Misconceptions

8.1 Misconception: int4 Quantization Always Means Everything Is int4

A model described as "int4" may only have int4 weights. Activations may remain bf16, and accumulation may happen in fp32.

More precise notation is:

[A16W4Acc32.]

8.2 Misconception: Lower Bitwidth Always Means Faster

Lower bitwidth reduces memory traffic, but it may add kernel overhead.

A quantized kernel can be slower if:

- it materializes large intermediate tensors,
- it performs expensive unpacking,
- it uses poor tile shapes,
- it fails to fuse scales,
- it converts low-bit values to high precision too early,
- it loses tensor core efficiency.

The true condition is:

[$\mathrm{T}_{\text{quantized kernel}} < T_{\text{baseline kernel}}$.]

8.3 Misconception: Weight-Only Quantization Is Less Work

Mathematically, weight-only quantization can be more work:

[Y =

$\mathrm{X}(\mathrm{s}_W W_q)$.]

Compared to unquantized:

[Y = XW.]

The quantized version adds scale and conversion logic. It is faster only when reduced weight bandwidth compensates for the extra operations.

8.4 Misconception: Dynamic Quantization Is Always Better Because It Quantizes More

Quantizing activations helps compute-bound workloads, but it can hurt memory-bound workloads. In LLM decoding, activation quantization may add overhead without reducing the dominant weight traffic enough.

8.5 Misconception: Per-Tensor Quantization Is Usually Enough

Per-tensor quantization can fail badly when different channels have different ranges. If one channel spans:

[[-1000,1000],]

and another spans:

[[-1,1],]

a shared scale makes the small channel nearly vanish. Per-channel or group-wise quantization helps preserve fidelity.

8.6 Misconception: Triton Eliminates the Need to Understand CUDA

Triton simplifies kernel writing, but performance still depends on CUDA concepts:

- memory coalescing,
- tiling,
- occupancy,
- register pressure,
- cache behavior,
- tensor core shapes,
- warp-level parallelism.

Triton is a productivity tool, not a replacement for hardware intuition.

8.7 Misconception: `torch.compile` Always Finds the Best Fusion

The lecture gives an example where `torch.compile` did not automatically fuse an int8 matmul followed by scaling. A hard-coded pattern was needed.

Compiler magic is useful, but the engineer must inspect generated kernels and profile real behavior.

9 Practical Takeaways

9.1 Use the Right Quantization Scheme for the Bottleneck

For compute-bound layers:

[A8W8 dynamic quantization]

may help because it reduces arithmetic cost.

For memory-bound LLM decoding:

[A16W8 or A16W4 weight-only quantization]

may help because it reduces weight bandwidth.

9.2 Fuse Rescaling Into the Kernel

Do not store an int32 matmul result if the next operation is a scale multiply. Instead, fuse:

[acc $\rightarrow$ scale $\rightarrow$ store .]

This reduces memory traffic and peak memory.

9.3 Do Not Materialize Dequantized Weights

Avoid:

[$\mathrm{W}_q \rightarrow \hat{W} \rightarrow X\hat{W}$.]

Prefer:

[X, $\mathrm{W}_q, s_W \rightarrow$ fused quantized matmul $\rightarrow Y$.]

9.4 Treat Batch Size One as a Special Case

LLM decoding at batch size one is not just a tiny GEMM. It is a vector-matrix product with different performance characteristics.

A kernel specialized for:

[M=1]

can be much faster than a generic GEMM kernel.

9.5 int8 Is Much Easier Than int4

int8 has better software and hardware support. int4 requires packing, unpacking, and careful layout. Expect int4 kernels to require more custom engineering.

9.6 Accuracy Requires Layer-Level Thinking

Some layers are more sensitive to quantization than others. A practical workflow is:

1. quantize one layer or group of layers,
2. measure speed,
3. measure quality,
4. identify sensitive layers,
5. keep sensitive layers at higher precision,
6. use GPTQ or QAT where needed,
7. repeat until reaching a Pareto-optimal trade-off.

The trade-off can be visualized as:
[speedup versus accuracy degradation.]
The goal is to move along the Pareto frontier.

10 Project or Experiment Ideas

10.1 Experiment 1: Dynamic Quantization With and Without Epilogue Fusion

Implement two versions of A8W8 linear inference:

1. int8 matmul that stores int32 output, then rescales in a second operation,
2. int8 matmul that fuses rescaling into the epilogue.

Measure:

- runtime,
- peak memory,
- global memory writes,
- kernel count,
- achieved occupancy.

Expected result:
[fused epilogue $\Rightarrow$ lower memory traffic and better runtime.]

10.2 Experiment 2: A16W8 Batch-Size-One Vector-Matrix Kernel

Compare:

- standard bf16 matmul,
- naive dequantize-then-matmul,
- Triton matmul with prologue cast,
- broadcast-sum `torch.compile` pattern,
- custom Triton vector-matrix kernel.

Use shapes such as:

[K=4096, N=4096.]

Measure tokens per second for an LLM-style linear layer.

10.3 Experiment 3: Batch Size Sweep for Weight-Only Quantization

Benchmark A16W8 for:

[$M \in 1, 2, 4, 8, 16, 32.$]

Observe when the operation transitions from memory-bound to compute-bound.

Plot:

[M versus runtime]

and:

[M versus achieved bandwidth.]

This experiment directly investigates why batch size greater than one is difficult.

10.4 Experiment 4: int4 Packing Layouts

Implement four packing layouts for int4 weights. For each layout, measure:

- load efficiency,
- number of bit operations,
- memory coalescing,
- runtime,
- L2 cache hit rate.

The goal is to understand why preserving contiguity along the kernel's reduction dimension matters.

10.5 Experiment 5: Quantization Granularity and Accuracy

Take a small transformer or MLP and compare:

- per-tensor quantization,
- per-channel quantization,
- group-wise quantization with group sizes (32), (64), (128), and (256).

Measure:

- validation accuracy or perplexity,
- model size,
- inference latency,
- scale metadata overhead.

Expected result:

[smaller groups $\Rightarrow$ better accuracy but more metadata.]

10.6 Experiment 6: Layer Sensitivity Study

For a transformer, quantize one layer at a time and measure degradation. Record which layers are most sensitive.

A possible table schema is:

[layer id, module type, bitwidth, perplexity change, latency change.]

This teaches the practical workflow used by quantization engineers.

10.7 Experiment 7: Compare PyTorch, Triton, and CUDA

Implement the same operation in three ways:

1. PyTorch eager,
2. `torch.compile` generated Triton,
3. custom Triton,
4. optional custom CUDA.

Inspect:

- generated kernels,

- number of memory reads and writes,
- dtype conversions,
- whether intermediate tensors are materialized.

This connects high-level Python code to GPU execution.

11 Visual Concept

Draw a diagram with two branches.

The first branch is dynamic quantization:

[X $\to X_q$, $W \to W_q$, $X_q W_q \to$ int32 accumulator $\to$ scale epilogue $\to Y$.]

The second branch is weight-only quantization:

[X stays bf16, W $\to W_q$, load packed weights $\to$ unpack/dequantize inside kernel $\to Y$.]

Emphasize that dynamic quantization attacks arithmetic cost, while weight-only quantization attacks memory bandwidth.

12 Visual Concept

Draw an SM-level view of a quantized matmul tile. Show global memory containing int8 or packed int4 weights, L2 cache, shared memory or registers, an int32 accumulator, scale values, and a final bf16 output store. Mark the bad path where int32 output is written to HBM before scaling, and the good path where scaling happens before the store.

13 Visual Concept

Draw the batch-size-one vector-matrix case. Show one activation vector (x) feeding many independent column reductions. Each column $(\mathrm{W}_{:,n}) produces one scalar (y_n). Label this as column - parallel execution.$

14 Final Mental Model

Quantization is not merely a numerical compression trick. On GPUs, it is a kernel design problem.

The mathematical promise is:

[fewer bits $\Rightarrow$ less memory traffic or cheaper arithmetic.]

The engineering reality is:

[fewer bits $\Rightarrow$ packing, unpacking, scaling, casting, accumulation, and compiler behavior.]

Dynamic quantization is best understood as a compute-throughput optimization:

[A8W8 ⇒ faster integer arithmetic]

when the layer is compute-bound.

Weight-only quantization is best understood as a memory-bandwidth optimization:

[A16W4 or A16W8 ⇒ fewer weight bytes loaded]

when the layer is memory-bound.

The most important kernel principle is:

[Never materialize an expensive intermediate if the next operation can be fused.]

For dynamic quantization, that means fusing scale multiplication into the matmul epilogue. For weight-only quantization, that means avoiding full dequantized weight tensors. For int4, that means designing kernels that handle packed values efficiently rather than expanding them too early.

Triton and `torch.compile` are powerful because they make many fused kernels easy to express and generate. But for very low-bit non-standard dtypes such as int4, custom CUDA or specialized kernels may still be necessary.

The final practical mindset is:

> First identify the bottleneck. Then choose the quantization scheme. Then design or select a kernel that preserves the theoretical advantage. Finally, verify with profiling and task-level evaluation.